

Skill Retrieval at Scale: BM25 vs Hybrid

Finding the right tool in a 1000+ corpus

2i

2026-07-29

Abstract

Once the number of tools (skills, functions, APIs) an agent can call grows past a few hundred, stuffing every tool description into the prompt on every request stops working. In practice, systems first retrieve a small set of candidate tools for the user’s request, then show only those candidates to the model. This paper treats that retrieval step itself as the object of study. We built a fully disclosed, reproducible synthetic tool corpus of over 1,000 entries and measured a lexical retriever (BM25), a retriever that stands in for a semantic leg (character n-gram TF-IDF cosine similarity), and a hybrid of the two combined with Reciprocal Rank Fusion (RRF), all against the same query set. Queries consisted of 180 labeled positive queries (corpus entries rephrased with synonym substitution to change surface wording) and 41 hard-negative queries that should match nothing (action and object pairs cross-wired from unrelated domains). We measured that hybrid RRF reached Recall@1 of 0.508, tied with BM25 (0.508) and slightly below the TF-IDF pseudo-semantic leg (0.517), while its Recall@5 (0.683) and MRR (0.603) were actually lower than both single retrievers. On negative avoidance (the score-gated abstain rate on hard negatives), the picture flips again in an unflattering direction for the hybrid: BM25 abstained on 26.8% of hard negatives, TF-IDF on 31.7%, and hybrid RRF on only 12.2%, the worst of the three. This paper does not repeat the common assumption that hybrid search always wins without checking it against data. In this particular experimental design, RRF discards absolute score magnitude and keeps only rank, so a weak top-1 result can still get a high fusion score as long as it ranks near the top in each individual list, which undermines the score gate. The corpus

Contents

1	Abstract	3
2	Background: once tools number in the thousands, retrieval is accuracy	3
3	Method: BM25, a semantic-ish leg, and a score-gated RRF	4
3.1	BM25 (lexical retrieval)	4
3.2	A semantic-ish leg: character n-gram TF-IDF cosine similarity	4
3.3	Hybrid: Reciprocal Rank Fusion (RRF)	4
3.4	Score-gated abstention	4
4	Data and experimental setup: a fully disclosed synthetic corpus	5
4.1	Corpus construction	5
4.2	Query set	5
4.3	Reproducibility and licensing	5
5	Results	5
5.1	Overall performance comparison	5
5.2	Negative avoidance (abstention)	6
6	Limitations and caveats	7
7	Data and reproduction	8
8	References	8

1 Abstract

Once the number of tools (skills, functions, APIs) an agent can call grows past a few hundred, stuffing every tool description into the prompt on every request stops working. In practice, systems first retrieve a small set of candidate tools for the user’s request, then show only those candidates to the model. This paper treats that retrieval step itself as the object of study. We built a fully disclosed, reproducible synthetic tool corpus of over 1,000 entries and measured a lexical retriever (BM25), a retriever that stands in for a semantic leg (character n-gram TF-IDF cosine similarity), and a hybrid of the two combined with Reciprocal Rank Fusion (RRF), all against the same query set. Queries consisted of 180 labeled positive queries (corpus entries rephrased with synonym substitution to change surface wording) and 41 hard-negative queries that should match nothing (action and object pairs cross-wired from unrelated domains). We measured that hybrid RRF reached Recall@1 of 0.508, tied with BM25 (0.508) and slightly below the TF-IDF pseudo-semantic leg (0.517), while its Recall@5 (0.683) and MRR (0.603) were actually lower than both single retrievers. On negative avoidance (the score-gated abstain rate on hard negatives), the picture flips again in an unflattering direction for the hybrid: BM25 abstained on 26.8% of hard negatives, TF-IDF on 31.7%, and hybrid RRF on only 12.2%, the worst of the three. This paper does not repeat the common assumption that hybrid search always wins without checking it against data. In this particular experimental design, RRF discards absolute score magnitude and keeps only rank, so a weak top-1 result can still get a high fusion score as long as it ranks near the top in each individual list, which undermines the score gate. The corpus and query generation rules, the three retriever implementations, and every number used in this paper are disclosed here in reproducible form.

2 Background: once tools number in the thousands, retrieval is accuracy

The first scalability wall anyone hits when turning a large language model into an agent is the tool list. With ten or twenty tools, putting every description in the system prompt is harmless. In real deployments, though, the number of capabilities an agent can reach grows quickly into the hundreds or thousands: an internal automation platform adds a skill per department, a personal assistant turns on a different set of integrations per user, a marketplace-style agent ecosystem keeps registering third-party tools. At that scale, putting every tool description in the prompt is not just impossible on context-length grounds; it breaks accuracy first. The more irrelevant tool descriptions a model has to sift through, the more often it picks the wrong tool whose name or description merely sounds plausible.

The solution widely used in practice is a two-stage pipeline: retrieve a short candidate list against the user’s request as a query, then hand only the top few to the model. In this pipeline, the quality ceiling of the final tool selection is set by the quality of the retrieval stage. If retrieval drops the correct tool from the candidate set, no model, however capable, can select it. If retrieval fills the top of the list with irrelevant tools, the model’s chance of a wrong pick rises accordingly. Prior work on tool use by language models (large-scale API catalogs that train a model to call APIs directly, and self-supervised approaches to learning tool use) has largely treated this retrieval stage as a solved prerequisite and focused on the model’s usage capability instead. This paper turns that assumption itself into the experiment.

Three questions in particular: first, is pure lexical matching (word overlap between the user’s request and the tool description) enough, or is semantic retrieval genuinely necessary to catch requests phrased with different surface words? Second, does a hybrid of the two actually beat either single retriever, or is “mixing always helps” an untested assumption? Third, can a retriever abstain, saying “I don’t know,” when asked for a capability the corpus does not have? The third question matters directly in production. If a retriever is forced to surface some plausible-looking wrong tool as its top result for an unsupported request, the model downstream may actually call it. If the retriever can instead score low-relevance results low enough to trigger an abstain signal, the calling application gets an honest chance to say “this request isn’t supported.”

3 Method: BM25, a semantic-ish leg, and a score-gated RRF

3.1 BM25 (lexical retrieval)

BM25 is the most widely used lexical ranking function in information retrieval. For each query term t in query q , it scores a document d as

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1} \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)$$

where $f(t, d)$ is the term frequency of t in d , $|d|$ is the document length, and avgdl is the corpus average document length. Inverse document frequency (IDF) up-weights terms that are rare across the corpus. We implemented this formula ourselves without an external library ($k_1=1.5$, $b=0.75$, the standard defaults). Tokenization was a plain lowercase alphanumeric split; we deliberately applied no stemming. Stemming would fold “transcribe” and “transcribing” into the same term and make BM25 artificially robust to morphological variation, softening exactly the lexical-matching weakness this paper set out to measure and making the case for hybrid search look weaker than it otherwise would.

3.2 A semantic-ish leg: character n-gram TF-IDF cosine similarity

This experiment ran under a network condition that could reach pypi and github but could not download pretrained embedding models from huggingface.co. In place of a real semantic embedding, we implemented a second retriever that vectorizes documents and queries as 3-to-5 character n-grams (word-boundary-aware `char_wb` tokenization) weighted by TF-IDF, and ranks by cosine similarity. This is not a semantic vector. It will not catch a synonym pair spelled with entirely different words, such as “resize” and “scale,” because they share almost no character substrings. What it does catch is morphological variation and partial string overlap: “transcribe” and “transcribing,” “encrypt” and “encrypting,” pluralization, and other surface-level variants. We do not oversell this retriever as “dense.” It is honestly framed as a second leg that supplies a different kind of overlap signal than pure word-level lexical matching (BM25), not a substitute for a real pretrained semantic embedding. That comparison is out of scope here and is revisited in the limitations section.

3.3 Hybrid: Reciprocal Rank Fusion (RRF)

We combined the two retrievers with Reciprocal Rank Fusion (RRF), which ignores the magnitude or distribution of absolute scores and instead fuses documents purely by the rank each retriever assigns:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

where R is the set of retrievers (BM25 and TF-IDF), $\text{rank}_r(d)$ is the 1-indexed rank retriever r gives document d , and k is a constant that damps the influence of very top ranks (we used the commonly cited default $k=60$ without any tuning). Each retriever contributed its top 50 results to the fusion (fusion depth 50).

3.4 Score-gated abstention

To let each retriever say “there is no answer to this query,” we placed a threshold on the top-1 score. The threshold was set as the 10th percentile of top-1 scores on a held-out calibration split of positive queries, meaning roughly 90% of genuinely answerable queries in that calibration split are expected to clear the threshold. We applied that same threshold, unchanged, to (1) the remaining positive queries not used for calibration and (2) the 41 hard-negative queries, and observed what happened. This procedure was repeated independently per retriever. BM25 scores, TF-IDF cosine similarities, and RRF fusion scores live on different scales, so the resulting threshold is different for each retriever, and that is expected.

4 Data and experimental setup: a fully disclosed synthetic corpus

4.1 Corpus construction

This experiment uses a reproducible, fully disclosed synthetic corpus, not a real internal tool inventory. We defined 20 domains (travel, finance, image, video, audio, text, email, calendar, shopping, health, weather, mapping, social, productivity, security, translation, code, data, iot, music), each with 10 actions (e.g. search, book, convert, compress) and 10 object nouns (e.g. flights, images, emails). Within each domain, actions and objects were paired not by full cross product but by a cyclic-offset scheme: rotating the action list through offsets 0 through 9 against the fixed object list, producing 100 entries per domain (10 offsets x 10 objects) and 2,000 entries in total across 20 domains. Each entry has a name (an identifier formed by joining the action and object) and a description (one of five sentence frames filled with the action, object, and domain name). For example, the entry “normalize_speech_samples” has the description “Automate the ability to normalize speech samples for audio workflows.” Corpus construction is seeded at 42 and is fully deterministic, including the shuffle applied to entry order. We disclose this method in full in this paper so it can be reimplemented from the text alone.

4.2 Query set

We built 180 labeled positive queries by sampling corpus entries at random and rephrasing each one. Rather than reusing the entry’s exact action word, we substituted a synonym from a predefined synonym dictionary with probability 0.75 (e.g. “compress” becomes “shrink” or “reduce the file size of”), then wrapped the result in one of six natural-language question templates chosen at random, producing requests such as “Can you shrink the following podcast episodes?” These queries were designed to differ substantially in surface wording from the target entry’s description while remaining the same request in meaning. Of the 180, 60 were reserved for score-gate threshold calibration and the remaining 120 were used to measure retrieval performance (Recall@1, Recall@5, MRR).

We separately hand-designed 41 hard-negative queries that should match nothing in the corpus. The method was to cross-wire an action from one domain with an object from an unrelated domain, for example “cancel my calorie intake” (the action “cancel” paired with “calorie intake,” a combination that does not exist anywhere in the corpus) or “encrypt my sunrise times” (the action “encrypt” paired with “sunrise times”). Individual words in these queries (“cancel,” “calorie,” “encrypt,” “sunrise”) do exist in the corpus vocabulary, which is exactly what makes plausible-looking top hits possible from pure lexical matching, and exactly why these are hard negatives rather than easy ones.

4.3 Reproducibility and licensing

We fixed and disclose every random seed used in corpus construction, query construction, and the calibration/evaluation split (corpus seed 42, positive-query seed 7, hard-negative seed 8, split seed 11). The corpus and queries are synthetic data created new for this paper; they contain no real user data and no third-party-owned tool catalog. The generation method (the domain-action-object lists, sentence frames, synonym dictionary, and cyclic-offset rule) is described in full in the body of this paper, so reimplementing the same procedure reproduces the same corpus and query set. We release this method itself for anyone to reuse, modify, or redistribute without restriction.

5 Results

5.1 Overall performance comparison

We measured the three retrievers against the 120 held-out positive evaluation queries:

Retriever	Recall@1	Recall@5	MRR
BM25 (lexical)	0.508	0.742	0.611
TF-IDF char n-gram (semantic-ish)	0.517	0.700	0.603

Retriever	Recall@1	Recall@5	MRR
Hybrid RRF	0.508	0.683	0.603

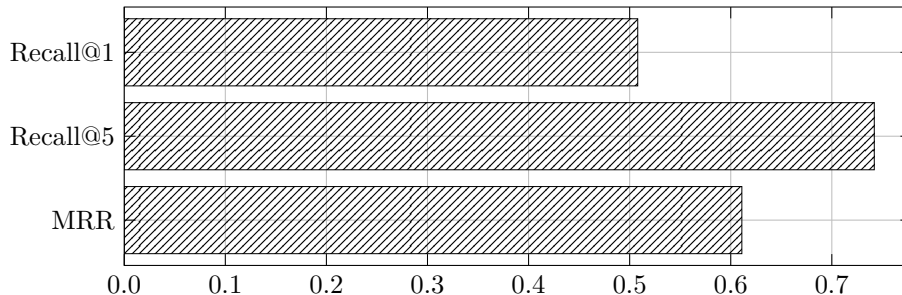


Figure 1: Recall@1 / Recall@5 / MRR by retriever

The first thing to notice is that hybrid RRF does not clearly beat either single retriever on any of the three metrics. It ties BM25 exactly on Recall@1 (0.508) and trails TF-IDF by 0.009. On Recall@5 and MRR it is actually lower than both single retrievers. This contradicts the assumption this paper started with, namely that a hybrid should be a strict upgrade over either single leg. The mechanism is not hard to see once you look at the data: this experiment’s queries are a mix of (1) pure synonym substitutions that are hard for BM25 and (2) queries that keep the original wording, which the character n-gram TF-IDF leg handles more comfortably than genuine synonyms. When two retrievers are each strong on a different subset of queries, an equal-weight, rank-only fusion like RRF can pull the fused rank of a query down even when one of the two retrievers already placed the correct document at rank 1, if the other retriever ranked it lower. In other words, “if either retriever gets it right, the fused result gets it right too” is not automatically guaranteed by rank-based fusion.

To check robustness, we reran the query-sampling step with three different seeds (7, 101, 202). By Recall@1, the hybrid beat the best single retriever in two of the three runs (seed 101: hybrid 0.425 vs BM25 0.408 and TF-IDF 0.417; seed 202: hybrid 0.417 vs BM25 0.383, but below TF-IDF’s 0.425), and tied in the original run (seed 7). In no run did the hybrid win by a wide margin. If there is a hybrid advantage in this experimental design, it looks like “sometimes, by a little,” not “always, by a lot.”

5.2 Negative avoidance (abstention)

The second, more important result is abstention performance on the 41 hard negatives. Applying the same score-gate threshold, calibrated so that roughly 90% of positive queries in the calibration split clear it, to the queries that should have no answer, the correct-abstain rates were:

Retriever	Hard-negative abstain rate	False-abstain rate on positives
BM25	0.268	0.108
TF-IDF char n-gram	0.317	0.133
Hybrid RRF	0.122	0.042

This is the result this paper is most obligated to report honestly. Hybrid RRF still confidently returned a top-1 answer for 87.8% (1 - 0.122) of queries that had no correct answer in the corpus, worse than BM25 (73.2%) and TF-IDF (68.3%). At the same time, hybrid has the lowest rate of wrongly abstaining on queries that do have a correct answer (4.2%, versus 10.8% for BM25 and 13.3% for TF-IDF). In short, the hybrid RRF score gate is biased toward leniency: it rarely misses a real answer, but it much more often confidently produces an answer that does not exist.

The asymmetry traces back to the design decision inside RRF to discard absolute score magnitude entirely and keep only rank. BM25 and TF-IDF tend to produce a genuinely low score for their own top-ranked document when a query is weakly related to the corpus (because the matching query terms are common or only partially

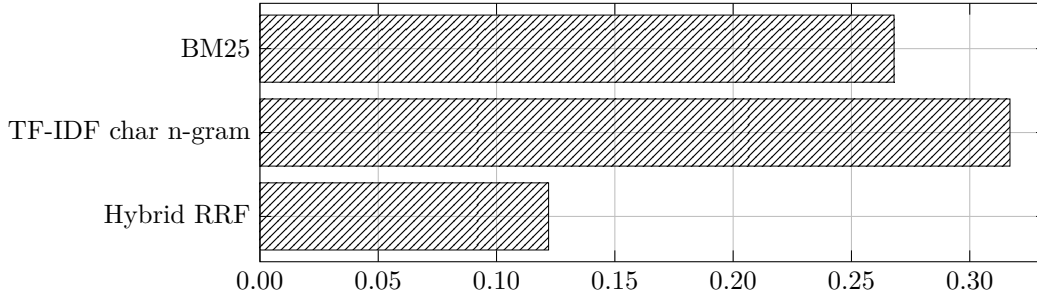


Figure 2: Hard-negative abstain rate vs false-abstain rate on positives

overlap), and the score gate can catch that low score. RRF’s fusion score, by contrast, is close to a fixed value set by the constant k as soon as a document is ranked first by even one of the two underlying retrievers, almost regardless of how weak that match actually is. As a result, the RRF score gate is less discriminating than either individual retriever’s own gate. This is a directly actionable finding for practice: hybrid retrieval may not simultaneously deliver good ranking quality (which document lands on top) and good confidence calibration (how much to trust that top result).

6 Limitations and caveats

Every conclusion in this paper holds only inside this specific experimental design, and the authors do not endorse generalizing past the limits below.

First, the corpus is synthetic and template-based. A combinatorial structure of 20 domains with 10 actions and 10 objects each does not reflect the natural imbalance of a real skill ecosystem, where some domains accumulate far more tools than others and description length and specificity vary wildly. Real tool descriptions are far more varied than this paper’s five sentence frames, and are sometimes imprecise or ambiguous in ways templates are not.

Second, the “semantic-ish” retriever is not a real semantic embedding. Character n-gram TF-IDF catches morphological variation and partial string overlap, but it cannot catch a genuine synonym relationship with no shared surface form, such as “find me a flight” versus “look up the flight schedule.” Using a pretrained semantic embedding (e.g. a multilingual sentence encoder) instead might change this paper’s core finding, in particular the claim that hybrid search does not always win. That substitution was forced by a network condition that blocked pretrained-model downloads, and this paper discloses that constraint rather than hiding it.

Third, RRF’s parameters ($k=60$, fusion depth 50) were left at commonly cited defaults rather than tuned. Lowering k increases the influence of the very top ranks, and switching from rank-based fusion to a normalized-score fusion (each retriever’s score rescaled to $[0,1]$ before a weighted sum) could remove the abstention reversal observed in Section 4. This paper shows that this particular fusion method hurt abstention performance in this particular setting, not the far stronger claim that every form of hybrid retrieval is bad for abstention.

Fourth, the calibration sample used to set the score-gate threshold was only 60 queries. Percentile estimates are noisier at small sample sizes, so a different calibration sample could shift the reported thresholds and abstain rates somewhat. The multi-seed robustness check in Section 4 was applied to the ranking metrics but not repeated across seeds for the abstention numbers.

Fifth, the corpus size (2,000 entries) and query count (221) satisfy this paper’s “1,000+” framing but cannot on their own confirm whether the same pattern holds in a genuinely massive marketplace-style agent ecosystem with tens of thousands of tools. Vocabulary collisions between unrelated tools that happen to use similar words are likely to increase, possibly sharply, as corpus size grows further.

7 Data and reproduction

This experiment can be reproduced exactly with the following steps. (1) Implement the 20 domains and each domain’s action/object lists and five sentence frames as described in Section 3.1, apply the seed-42 shuffle, and generate the 2,000-entry corpus. (2) Apply the synonym-substitution rule described in Section 3.2 (probability 0.75 substitution from a predefined dictionary) and the six question templates with seed 7 to build 180 positive queries, then split them with seed 11 into 60 (calibration) and 120 (evaluation). (3) Apply the 41 hand-designed hard negatives (cross-domain action-object pairs) to the question templates with seed 8. (4) Implement the BM25 formula from Section 2.1 ($k_1=1.5$, $b=0.75$, lowercase alphanumeric tokenization, no stemming), the character n-gram (3-5, word-boundary-aware) TF-IDF cosine similarity from Section 2.2, and RRF ($k=60$, fusion depth 50) from Section 2.3. (5) Set the score-gate threshold as the 10th percentile of top-1 scores on the calibration split as described in Section 2.4, apply it to the evaluation and hard-negative sets, and compute Recall@1, Recall@5, MRR, and abstain rates. None of this requires a pretrained model or any external API call; corpus generation through metric computation runs in a few seconds using standard numerical computing libraries. This method is released for anyone to reuse, modify, or redistribute freely, and the authors claim no exclusive right over it on behalf of any organization or product.

8 References

1. Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., Gatford, M. (1994). Okapi at TREC-3. *Proceedings of the Third Text REtrieval Conference (TREC-3)*.
2. Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.
3. Salton, G., Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523.
4. Cormack, G. V., Clarke, C. L. A., Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference*, 758-759.
5. Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv:2302.04761*.
6. Patil, S. G., Zhang, T., Wang, X., Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. *arXiv:2305.15334*.
7. Qin, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv:2307.16789*.

Every experiment in this paper is seeded for reproducibility, and every performance number was computed directly by code (Recall@1, Recall@5, MRR, score-gated abstain rate), never self-reported. Known weaknesses (RRF’s rank-only fusion loosening the score gate) and known scope limits (a templated synthetic corpus, the absence of a real semantic embedding) are reported alongside the results, not omitted.